musiccloud Resolver-Flow

Architecture and Data Flow Through the Resolver

musiccloud Backend
Resolver Subsystem

Version `1.1.0+a1bcf62f-dirty` · 2026-05-01

Abstract

The resolver is the translation layer between music inputs (URLs, free-text queries, and genre queries) and a unified, cross-service response that exposes the same recording on Spotify, Apple Music, Tidal, Deezer, YouTube, and additional services. This document describes the architecture of the resolver and the data flow through its layers: ingress endpoints, dispatcher, cache tiers, adapter calls, the cross-service expansion, and the persistence stage, which terminates in a stable share short-link. The aim is a reference for the architecture and internal behaviour of the resolver.

Keywords: resolver, cache, adapter, cross-service expansion, disambiguation, persistence, share short-link, ISRC.

Contents

1 What this document covers **2**

2 How a request enters the system **2**
2.1 The authenticated endpoint . . . 2
2.2 The unauthenticated endpoint . . . 2
2.3 How a token is issued . . . 3
2.4 Adjacent endpoints . . . 3

3 What the dispatcher decides **3**

4 The path of a URL **5**
4.1 Cleaning up the URL . . . 5
4.2 The three cache tiers . . . 5
4.3 Identifying the source . . . 5
4.4 Fetching metadata . . . 7

5 The path of a free-text search **9**
5.1 The first pass . . . 9
5.2 Returning with a chosen candidate . . . 11

6 Genre discovery **13**
6.1 Browse mode . . . 13
6.2 Search mode . . . 15

7 Structured search **17**
7.1 What it's for 17
7.2 Form 17
7.3 Examples 17
7.4 What comes back 17
7.5 What's not allowed 17

8 Cross-service expansion **17**

9 Persistence **20**
9.1 The canonical write 20
9.2 Auxiliary writes and preview refresh 22

10 The cache strategy in one place **24**

11 Confidence thresholds **24**

12 Design decisions at a glance **24**

1 What this document covers

A user who pastes a streaming link into musiccloud expects a share page that opens the same recording on Spotify, Apple Music, Tidal, Deezer, YouTube, and a number of further services. Producing that translation is the resolver's responsibility. The present document describes its structure and the data flow through its individual layers.

The resolver handles five kinds of input. Each follows a slightly different path through the system, but once a result is produced, every path converges at the same point: the database, the share link, and the response returned to the user. What happens between input and result is the interesting part. Caches intercept recurring queries, and adapters mediate between the streaming services. A series of fallbacks keeps the system operational even when an individual service is temporarily unavailable.

2 How a request enters the system

The resolver is reachable through two HTTP endpoints. Both feed into the same dispatcher and the same downstream pipeline; they differ in who is allowed to call them, how often that may happen, and the form in which they communicate with the caller.

2.1 The authenticated endpoint

`POST /api/v1/resolve` is the full-featured entry point. Every request must authenticate, either with an `X-API-Key` header for internal callers or with an `Authorization` header of the form `Bearer <jwt>` for external clients holding an OAuth token. A per-IP rate limit applies in addition, because a stolen key remains a key.

The request body carries exactly one of two fields: either `query` with the URL or search term entered by the user, or `selectedCandidate` when the client is completing a previously initiated disambiguation.

Table 1: *Request body for `POST /api/v1/resolve`.*

Field	Type	Use	Meaning
`query`	string	first request	User input, at most 500 characters.
`selectedCandidate`	string	follow-up	Identifier of a previously offered candidate, at most 200 characters.

The response shape depends on the outcome. On a single unambiguous match the resolver returns a success payload that carries the metadata, the cross-service links, and the share short-link. When the best candidate of a free-text search is not sufficiently confident, it returns a list of suggestions instead. The client renders that list, takes the user's pick, and submits it as a follow-up request. A genre query receives a result list intended for display only; nothing is written to the database until the user clicks one of the entries.

On error the response carries an error code mapped to an HTTP status: `400` for malformed input, `401` for invalid credentials, `404` for a URL whose source could not be identified, `408` for a backend timeout, `429` on rate limit, `500` on an unexpected failure, and `503` when a service required by the resolver is not configured.

2.2 The unauthenticated endpoint

`GET /api/v1/resolve` addresses tools that cannot easily carry a bearer token: Apple Shortcuts, single-line curl invocations, bookmarklets, and similar script integrations. With no authentication in place, the per-IP rate limit alone protects against abuse. It is therefore set tighter than at the authenticated endpoint.

Table 2: Parameters for `GET /api/v1/resolve`.

Parameter	Type	Required	Meaning
`query`	string	yes	URL or search term, at most 500 characters.
`format`	string	no	`json` (default) returns the full response. `text` returns only the share short-link as plain text, suitable for use in a pipe.

A free-text search that resolves to multiple suggestions cannot be served by this endpoint, because a stateless single request permits no second step. In that case the response is `400`. The remaining error codes match those of the authenticated endpoint; only `401` drops out, as no authentication exists to fail.

A typical curl invocation reads as follows:

```
curl -G 'https://api.musiccloud.example/api/v1/resolve' \
  --data-urlencode 'query=https://open.spotify.com/track/...' \
  --data-urlencode 'format=text'
```

The result is the share short-link, ready to be pasted into a message or piped into another command.

2.3 How a token is issued

Before an external client may call the authenticated endpoint, it exchanges its credentials for a JWT. This happens at `POST /api/auth/token` via the OAuth 2.0 client-credentials flow.

Table 3: Fields of the token request.

Field	Type	Meaning
`client_id`	string	Client identifier registered with musiccloud.
`client_secret`	string	The associated secret.
`grant_type`	string	Must be exactly `client_credentials`.

The response carries three fields: `access_token`, the JWT to be sent in subsequent requests as an `Authorization` header of the form `Bearer <token>`; `token_type` with the fixed value `Bearer`; and `expires_in` with the validity in seconds, currently one hour. When the token expires, the client simply requests a new one with the same credentials. A refresh-token mechanism is intentionally absent.

2.4 Adjacent endpoints

Two further endpoints belong to the resolve experience but lie outside the scope of this document:

- `GET` `/api/v1/share/{shortId}` returns the share page for a previously resolved entry, looked up by short identifier.
- `GET` `/api/v1/share/{shortId}/preview` refreshes an expired or missing preview URL on demand. The audio player in the frontend calls this endpoint as soon as it detects that a cached preview is no longer playable.

3 What the dispatcher decides

Both endpoints feed into the same first step: determining which shape the input has. Six outcomes are possible, and the dispatcher checks them in order.

If the request body carries the `selectedCandidate` field instead of `query`, this is the second step of an ongoing disambiguation. The candidate identifier carries enough information to resume where

the first request ended. The dispatcher passes the request directly to the matching handler; this path exists only at the authenticated endpoint.

If the input is literally `genre: ?`, the dispatcher returns the prepared genre browse grid without resolving anything. This grid is what appears on the discovery page; clicking a tile subsequently triggers a regular resolve call.

If the input begins with the keyword `genre:`, the request is a structured genre search, e.g. `genre: jazz, count: 20`. The dispatcher hands it to the genre layer, which parses the syntax and forwards requests to the discovery backend.

If the query starts with `title:`, `artist:`, or `album:`, it is handled as a structured search and routed through the same disambiguation path as free-text, but with a pre-parsed `SearchQuery` instead of a raw string. See “Structured search”.

If the input parses as a URL, the dispatcher classifies it further: album URLs go to the album resolver, artist URLs to the artist resolver, everything else to the track resolver. The three resolvers share most of their machinery and differ chiefly in the identifiers they operate on.

If none of these rules applies, the resolver treats the input as a free-text search. The authenticated endpoint can answer with either a single match or a suggestion list; the unauthenticated endpoint accepts only the unambiguous case.

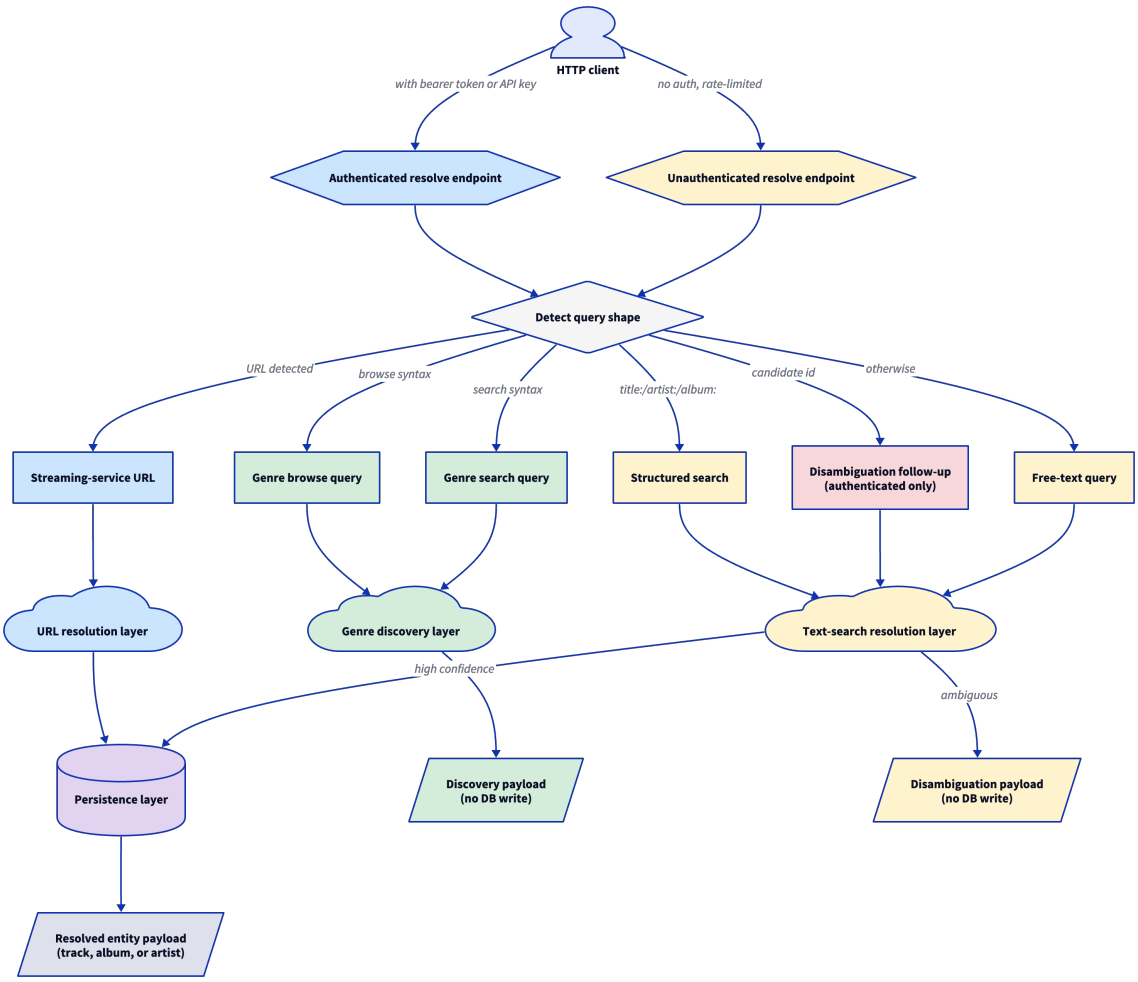

Figure 1: *The six possible exits of the dispatcher.*

4 The path of a URL

URLs are the most common input form and follow the most elaborate path. Album and artist URLs branch off early into their own resolvers. The principal mechanics, however, live in the track path, which is what this section follows.

4.1 Cleaning up the URL

The first step is to clean up the URL. URLs from the open web are rarely tidy. They carry traces of marketing campaigns, social-media buttons, and analytics tools in the form of additional parameters. Two requests pointing to the same Spotify track but differing in the value of `utm_source` would, without this cleanup, produce two distinct cache entries. Stripping the parameters is what makes the cache useful in the first place.

Some providers conceal the actual URL behind a redirect, e.g. through `link.deezer.com` or `on.soundcloud.com`. The resolver follows such redirects with an HTTP HEAD request to obtain the canonical URL, then strips parameters again. The redirect target frequently re-injects its own parameters, so a single pass of parameter stripping is not enough.

4.2 The three cache tiers

Before the resolver contacts an external service, it consults three caches in fixed order.

The first lookup uses the cleaned URL. If the same track has been resolved before via exactly this URL, the database row is still valid. Since the cache architecture migration, canonical entries no longer expire; they are merely augmented as new adapters become available.

The second lookup uses the original short link as an alias. When a URL was resolved from a short link, the resolver stores the original link as an additional key. The next request for the same short link hits the cache without paying for the redirect again.

The third lookup is keyed by the ISRC[1], the industry identifier of the recording. This check runs only after the metadata fetch from the source service, since the ISRC is not known earlier. The same recording can be reached through URLs of different services; the ISRC lets the resolver detect that condition and reuse the existing entry.

4.3 Identifying the source

If none of the three cache tiers hits, the resolver must determine which service the URL belongs to. It queries every installed plugin, including those that are currently disabled. This appears counter-intuitive at first, but it is intentional: a user pasting a Tidal link learns, on failure, that Tidal is disabled, instead of receiving the generic “not a music link” message. Which plugins are active is a configuration matter; which link the user pastes is not. The error message is meant to reflect that distinction.

If no plugin recognises the URL, the resolver responds with the generic message “not a music link”. If a disabled plugin recognises it, the response is the more specific “service unavailable”.

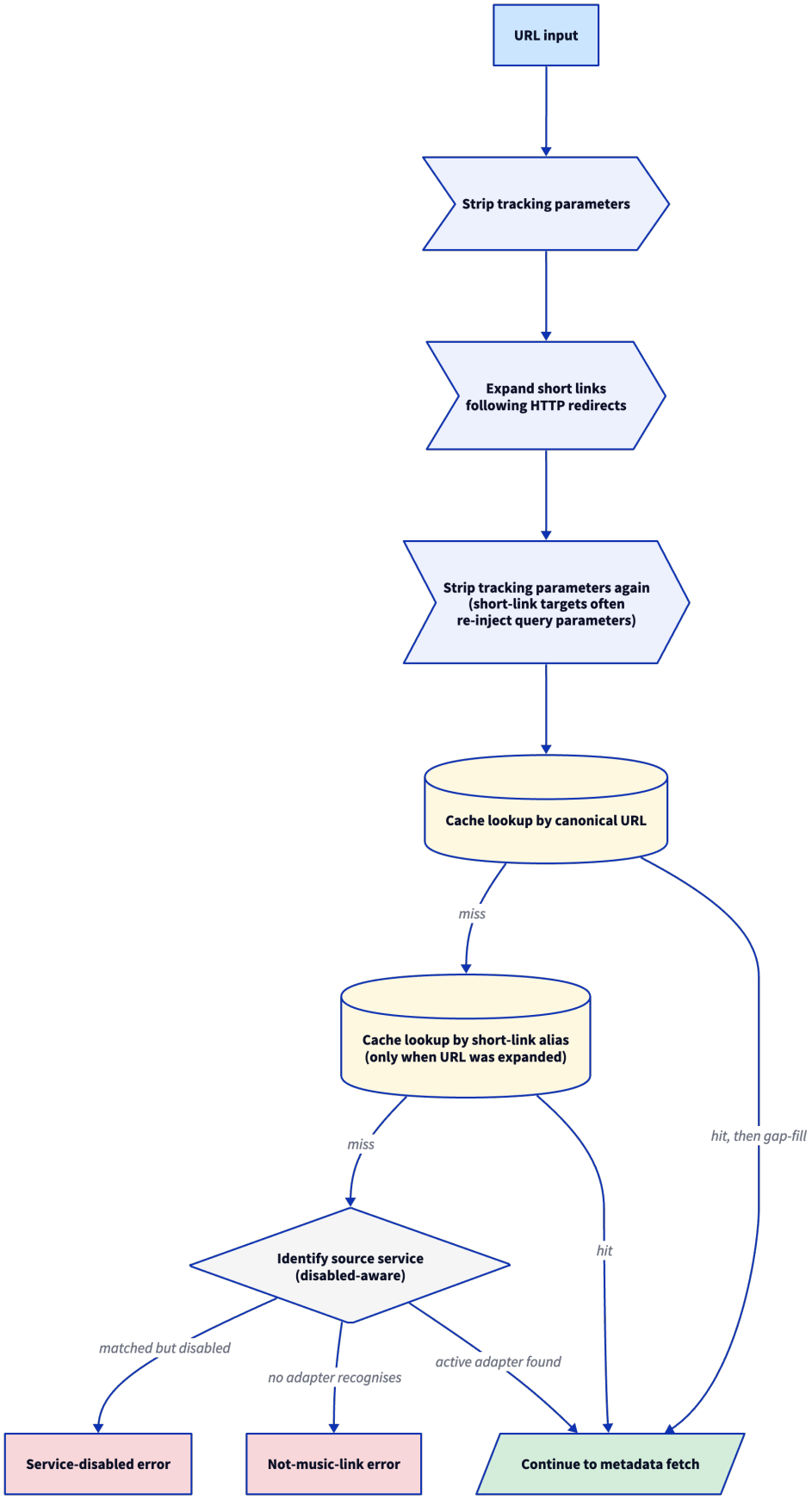

Figure 2: *URL path: cache tiers and source identification.*

4.4 Fetching metadata

Once the source service is known, the responsible adapter extracts the track identifier from the URL and queries the service for metadata: title, artist, album, ISRC, duration, artwork, and the URL of the audio preview.

Adapters fall into two groups. The first has API access: Spotify, Apple Music, Deezer, Tidal, and most further providers. They return structured responses. The second works without access and reads the public web page of the service. When such an adapter runs without configured credentials, the resolver fetches the page using a User-Agent identifying as a social-network crawler and parses the Open Graph meta tags from the returned HTML. This detour is necessary because many modern web pages render server-side only when they recognise a known crawler; for a plain browser they ship an empty JavaScript shell that loads content asynchronously.

Once the source metadata is in hand, the cross-service expansion runs in parallel against every other registered adapter; the section below describes it in detail. Two cleanup steps follow before the response is assembled.

The first concerns the artwork. Some services do not return artwork in their track API. If the source carries none but an ISRC is known, the resolver fetches artwork from another service's match for that ISRC. Spotify and Apple Music sit at the front of this fallback chain because both consistently return high-quality artwork.

The second concerns the audio preview. Previews come in two flavours: permanently valid CDN URLs on the one hand, and signed URLs with a short lifetime on the other. A signed URL that works today can return `403` tomorrow. The resolver therefore always selects the most stable preview available. In practice this means a Deezer preview displaces a Spotify preview in every case, even when Spotify was the original source.

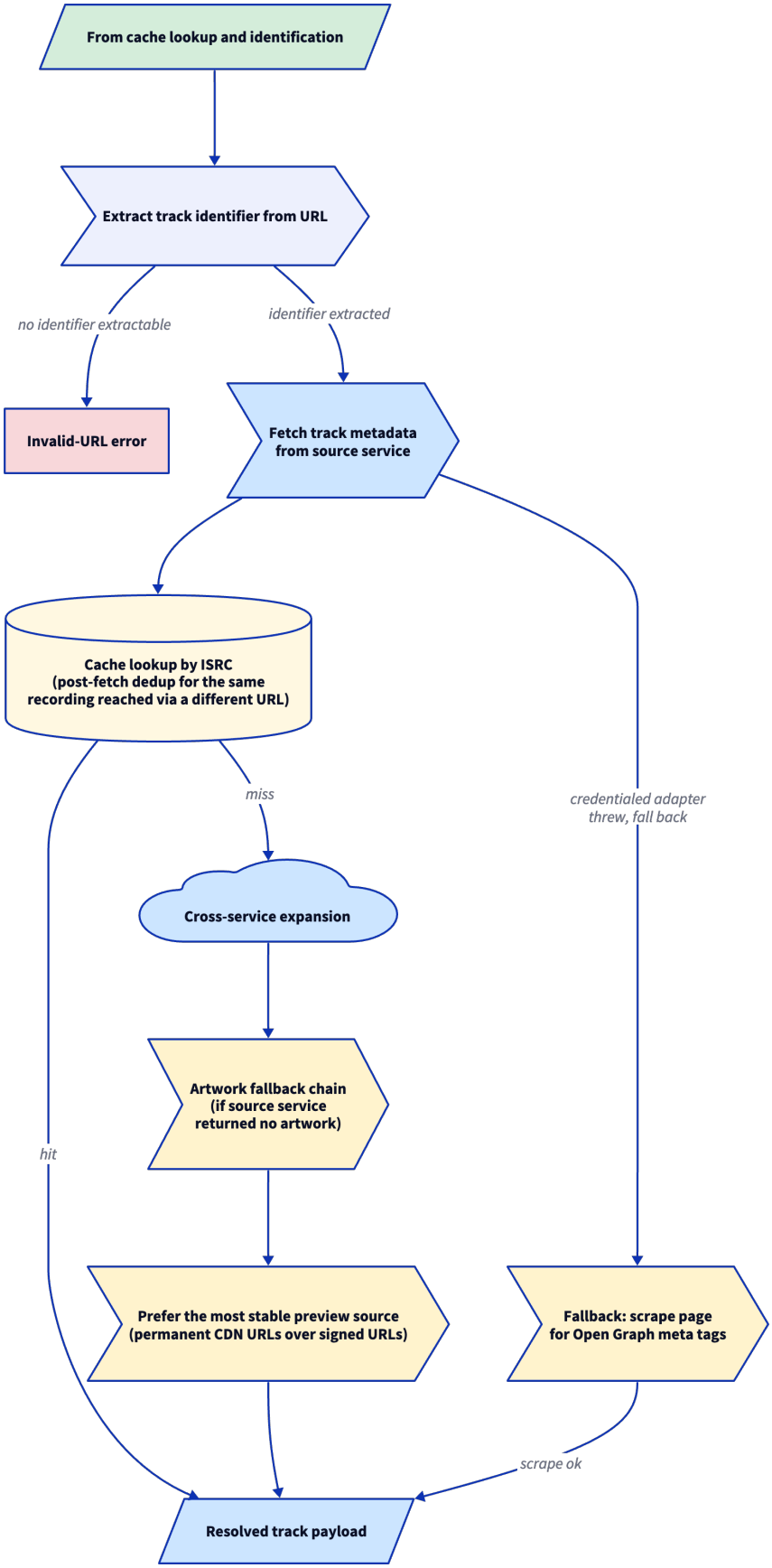

Figure 3: *URL path: metadata fetch and cross-service expansion.*

5 The path of a free-text search

The free-text search captures everything that is neither a URL nor a genre query. A user typing `daft punk around the world` expects a track as the result.

At this point the two endpoints diverge. The authenticated endpoint can hold a dialogue with the client: when the search yields no clear match, the endpoint returns a list of suggestions, from which the client picks one. The unauthenticated endpoint knows no dialogue. It responds only when the result is unambiguous and otherwise returns `400`.

5.1 The first pass

The resolver walks the active adapters in a fixed order. ISRC-rich sources with broad catalogue coverage (Deezer, Apple Music, Tidal) lead. Spotify drops to the end of the main chain and serves only as a fallback after ISRC misses. The remaining adapters form the safety net.

Adapters split here into two groups as well, this time by the behaviour of their search API. Some return only the single best result with a confidence score; others return a sorted list with scores per match. Only the second variant makes disambiguation possible at all.

When the adapter returns a single result, the resolver can either accept it directly, provided it is convincing enough, or move on to the next adapter. When the adapter returns a list, the resolver inspects the top hit. If its confidence exceeds the threshold for direct selection, the resolver accepts it without querying the user. Otherwise the resolver passes the list back to the client.

Three thresholds govern this behaviour. The first decides whether the top candidate may skip disambiguation. The second filters candidates whose confidence is too low to be worth showing the user at all. The third caps the list size so an ambiguous query cannot generate an arbitrarily long response.

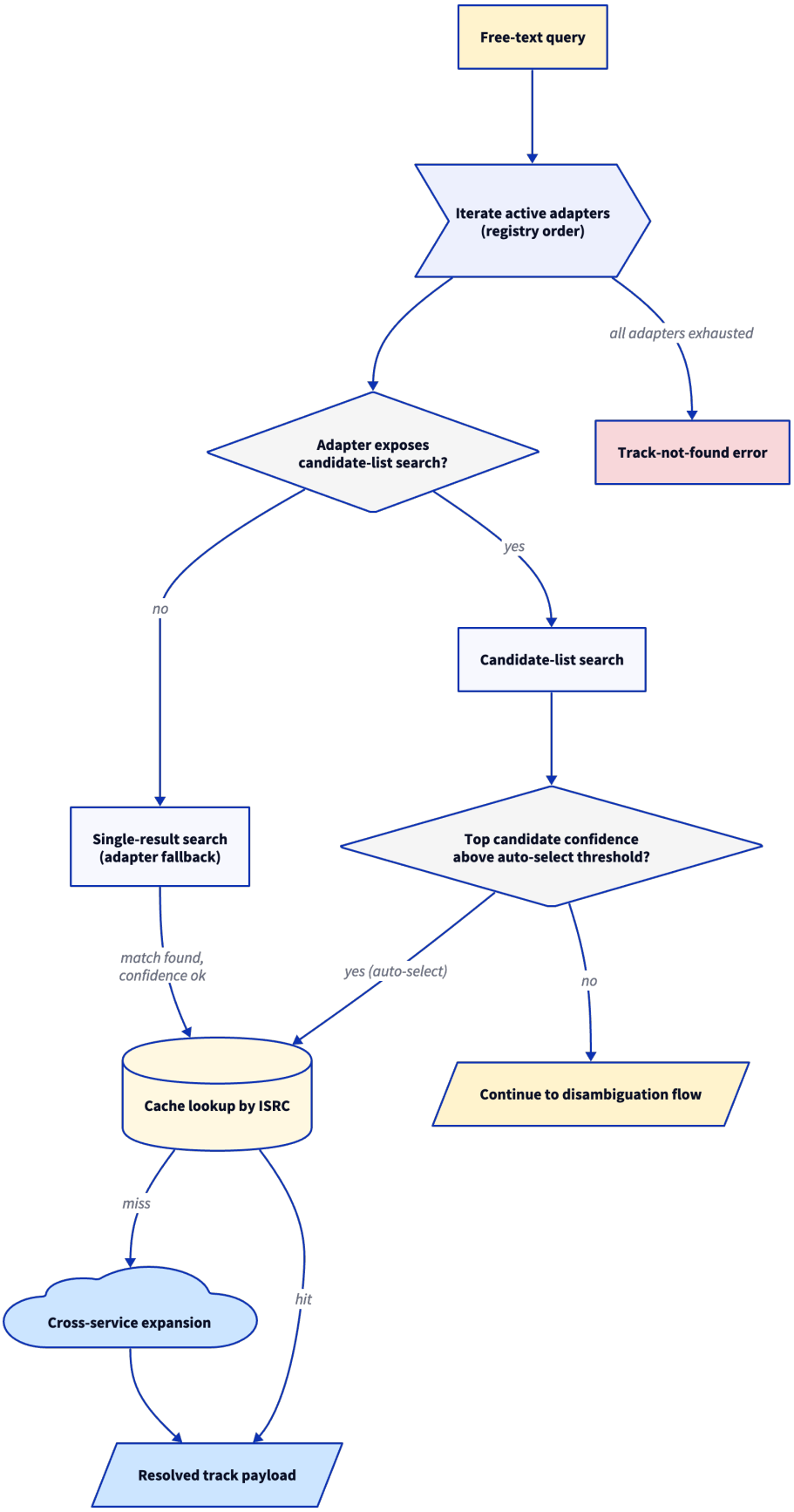

Figure 4: *Free-text search, first pass.*

5.2 Returning with a chosen candidate

Disambiguation is a two-step dialogue. On the first request the resolver returns the suggestion list, the client renders it, and once the user has made a selection, the client sends a second request carrying the `selectedCandidate` field. This second request follows a different path: the resolver fetches the full metadata of the chosen candidate from the source service, checks the cache by ISRC, and then runs the cross-service expansion, exactly as for a URL resolution.

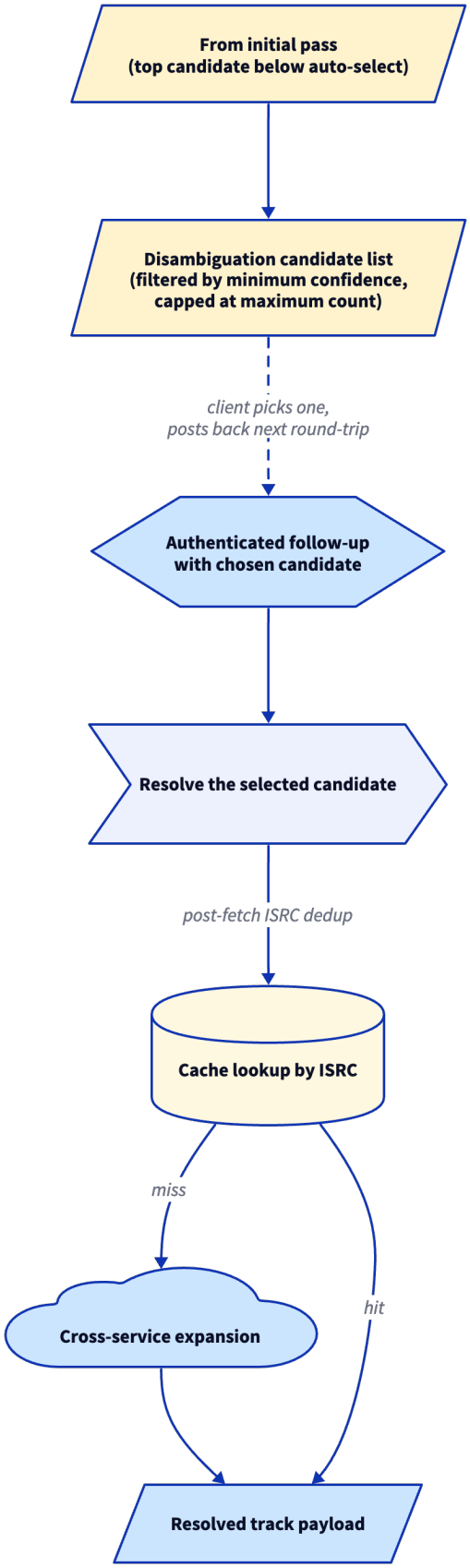

Figure 5: *Free-text search, disambiguation follow-up.*

If no adapter produces a usable result, the resolver responds with “track not found”. In practice this happens most often on typos, very obscure recordings, and inputs the user did not intend as a search for a music title in the first place.

6 Genre discovery

If you are searching for a specific track, see “Structured search”. This section is about browsing by genre.

The genre path is the only part of the resolver that writes nothing to the canonical database. It returns a selection of suggestions only. When the user clicks one, the click target flows back through the URL or text path into the resolver, where it is persisted regularly.

The discovery backend rests on community-assigned tags and therefore knows thousands of subgenres, far beyond what the major streaming catalogues directly offer. Trance, Krautrock, Shoegaze, Bebop, Vaporwave, Dungeon Synth, and the like are found only there. Without the matching API key, the entire genre path is unusable; this is one of the few places where a feature genuinely depends on a single external service.

6.1 Browse mode

When the user submits the literal request `genre: ?`, the resolver returns the prepared genre browse grid: a few hundred tags, filtered against a deny list (countries, decades, instruments, and personal collection labels are not genres) and canonicalised for spelling, so that the same genre name does not appear multiple times. The full grid is held in memory for 24 hours; the expensive preparation runs only once per backend restart.

Each tile in the grid points to a relative URL that targets a separate endpoint. That endpoint renders the cover image on first access and stores it in the cache; all subsequent accesses receive the prepared image.

```
GET /api/v1/resolve?query=genre%3A+%3F
```

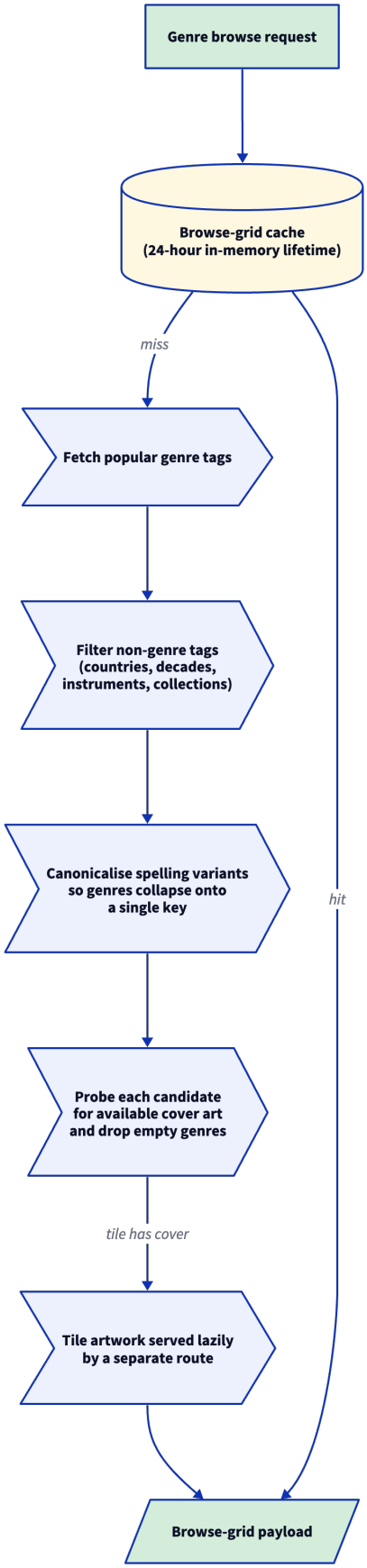

Figure 6: *Genre discovery, browse mode.*

6.2 Search mode

When the input begins with the keyword `genre:`, the resolver parses it as a structured form. The grammar is deliberately simple and accepts queries such as the following:

```
genre: jazz
genre: rock | metal
genre: hip hop, count: 20
genre: ambient, tracks: 30, vibe: mixed
genre: classical | baroque, albums: 12, artists: 8
```

The available fields are:

Table 4: *Fields of a genre search.*

Field	Meaning
`genre`	One or more genres separated by `|`. Required.
`tracks`	Number of tracks requested. Default: 10.
`albums`	Number of albums requested.
`artists`	Number of artists requested.
`count`	Shorthand to set the same count for all three categories.
`vibe`	`hot` prefers popular hits (default); `mixed` interleaves popular, mid-tier, and lesser-known matches.

Multiple genres in a single query are interpreted as an OR condition: `genre: jazz | bebop` returns matches tagged under either name. Unknown fields or invalid numbers raise an error; the parser does not silently drop anything.

The two values for `vibe` differ in how items are drawn from the per-genre data. `hot` takes the top of the chart, optionally evenly spaced across the top ranks so that not every match comes from the same calendar week. `mixed` draws a stratified sample from the upper, middle, and lower thirds; the result leans noticeably toward discovery rather than recognition.

Once the data is in hand, the resolver fills missing artwork from the persistent image cache. Images that surface here for the first time are also written to the cache, so subsequent requests load nothing more from upstream.

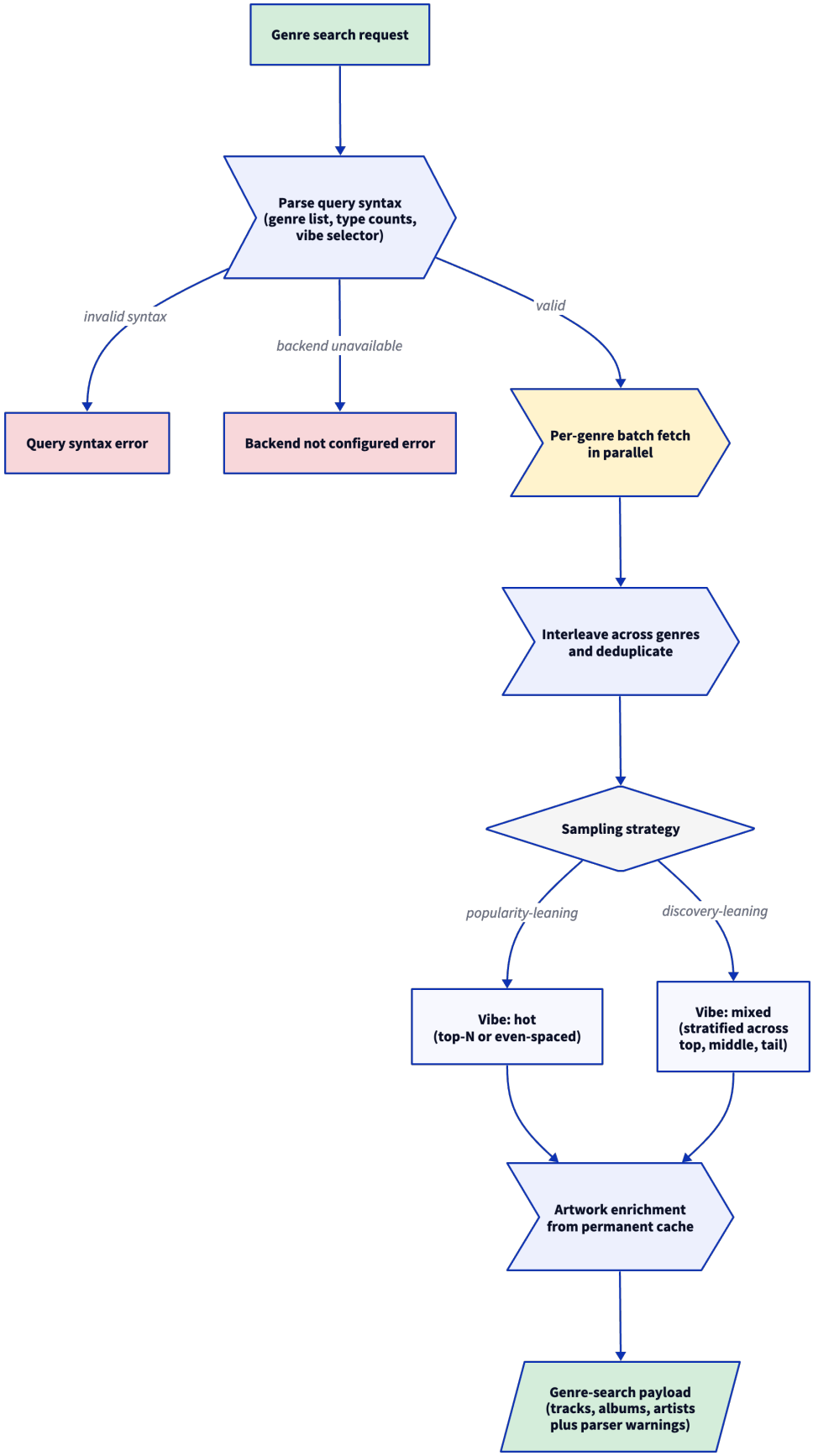

Figure 7: *Genre discovery, search mode.*

7 Structured search

7.1 What it's for

When the user has a specific track in mind and knows the artist, structured search is more precise than free text. Instead of asking the resolver to guess from a single string, the resolver receives the title, the artist, and optionally the album as separate fields. The adapters that support field operators (Spotify, Deezer, MusicBrainz) use them; the others fall back to a richer concatenated term.

7.2 Form

A structured query starts with one of three keys: `title:`, `artist:`, or `album:`. Fields are comma-separated, keys are case-insensitive, and whitespace around `:` and `,` is collapsed. At least one of `title:` or `artist:` must be present — `album:` alone is too weak to drive a track search reliably and is rejected.

The optional `count:` modifier (1–10) caps the disambiguation candidate list.

7.3 Examples

- `title: The Killing Moon, artist: Echo & The Bunnymen` — the classic case: title and artist both known.
- `artist: Radiohead` — artist only; the system returns a disambiguation list of well-known Radiohead tracks.
- `title: Karma Police, artist: Radiohead, album: OK Computer, count: 5` — all three fields set, list capped at 5.
- `title: Bohemian Rhapsody` — title only; resolves directly when confident, otherwise returns candidates.

7.4 What comes back

The same response shapes as free text: either a single resolved track (when confidence clears the threshold described in "Confidence thresholds") or a disambiguation list with up to `count` candidates (otherwise capped at `MAX_CANDIDATES`, currently eight). Send the picked candidate back as `selectedCandidate` to land on the share page.

7.5 What's not allowed

Keys that belong to genre-discovery (`genre:`, `tracks:`, `albums:`, `artists:`, `vibe:`) are explicitly rejected here with a 400 response and a directive error message pointing to the right query shape. This keeps the two modes cleanly separated: structured search is resolver mode (one track), genre-discovery is browse mode (a list to look through).

8 Cross-service expansion

This step is what makes the resolver actually useful. The user pasted a link from a single service, but the share page should open Spotify, Apple Music, Tidal, Deezer, YouTube, and the remaining services. The cross-service expansion is the mechanism by which the resolver locates the same recording on each further registered service.

Once the source is known, every other adapter runs in parallel against it. A per-adapter timeout prevents a single blocking adapter from holding up the whole response; without that timeout only the general HTTP timeout would kick in, which is set considerably longer.

Two strategies are tried per adapter, in decreasing order of evidence.

The stronger strategy is identifier lookup. If the adapter supports ISRC and one is available for the source, the adapter asks, in essence: "Do you have exactly this recording?" The result is unambiguous, hit or miss. A hit is scored at full confidence, since the ISRC is the strongest cross-service track identifier.

The weaker strategy is search by title and artist. The adapter runs the fuzzy search exposed by its API. The confidence score comes from that API; quality varies markedly between providers. Spotify's relevance ranking is reliable; the rest tend to look more like rough estimates.

Matches below a minimum quality are filtered out before the user sees them. An exception is made for synthetic search-fallback links, i.e. links pointing not directly at the match but at the service's search page. These are retained so that the response contains at least one option per service.

Two derivations follow. From every successful YouTube match the resolver derives the corresponding YouTube Music link by swapping the hostname and reusing the video or channel identifier. A separate search is unnecessary. If no direct YouTube match is found, the resolver inserts a synthetic link to the YouTube search, ensuring the YouTube entry is never absent from the response.

Album and artist searches follow analogous logic with their own match strategies. For albums, the first attempt is by UPC[2], followed by album inference from track ISRCs (a sample of tracks is looked up at the target service, and the most frequently reported album wins), and finally text search. For artists, the resolver uses its own confidence formula that distinguishes exact matches, "The"-prefix variants, high similarity, substring hits, and pure fuzzy matches.

All thresholds live in a central constants module. This central location keeps the confidence behaviour traceable in one place.

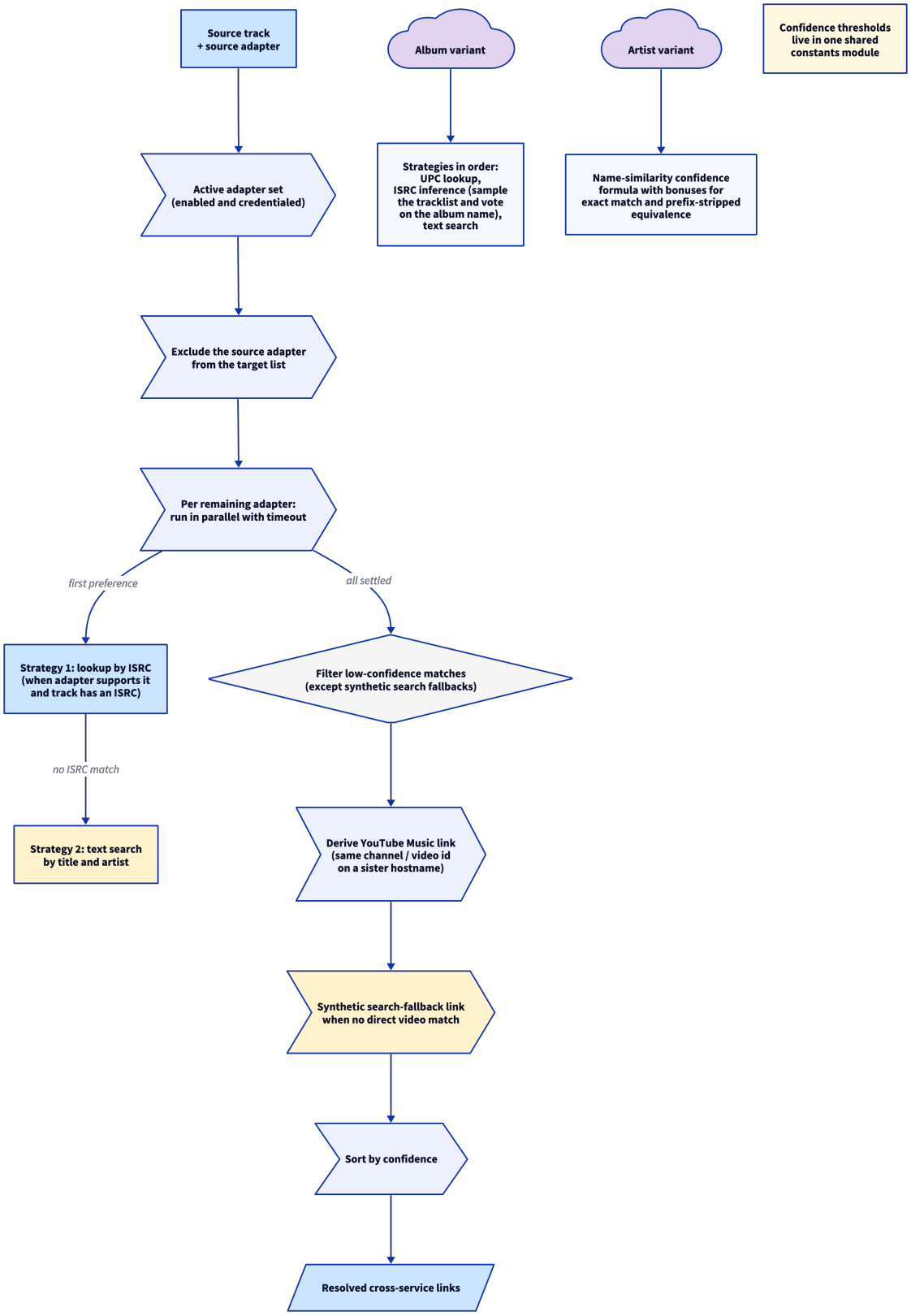

Figure 8: *Cross-service expansion.*

9 Persistence

Once a resolution result is available, the persistence layer writes it to the database in a fixed order. The route layer and the crawler path share this layer, which is what gives both callers an identical understanding of what “persisted” means.

9.1 The canonical write

The canonical track row, with its cross-service links, comes first. The resolver deduplicates either by ISRC or by source URL. If a matching row exists, its metadata is updated; otherwise a new row is inserted with fresh identifiers. Cross-service links are upserted on the (entry, service) pair, so that a repeated resolve of the same track is idempotent. The share short-link identifier is created only when the track itself is new, so the share link remains stable across re-resolves.

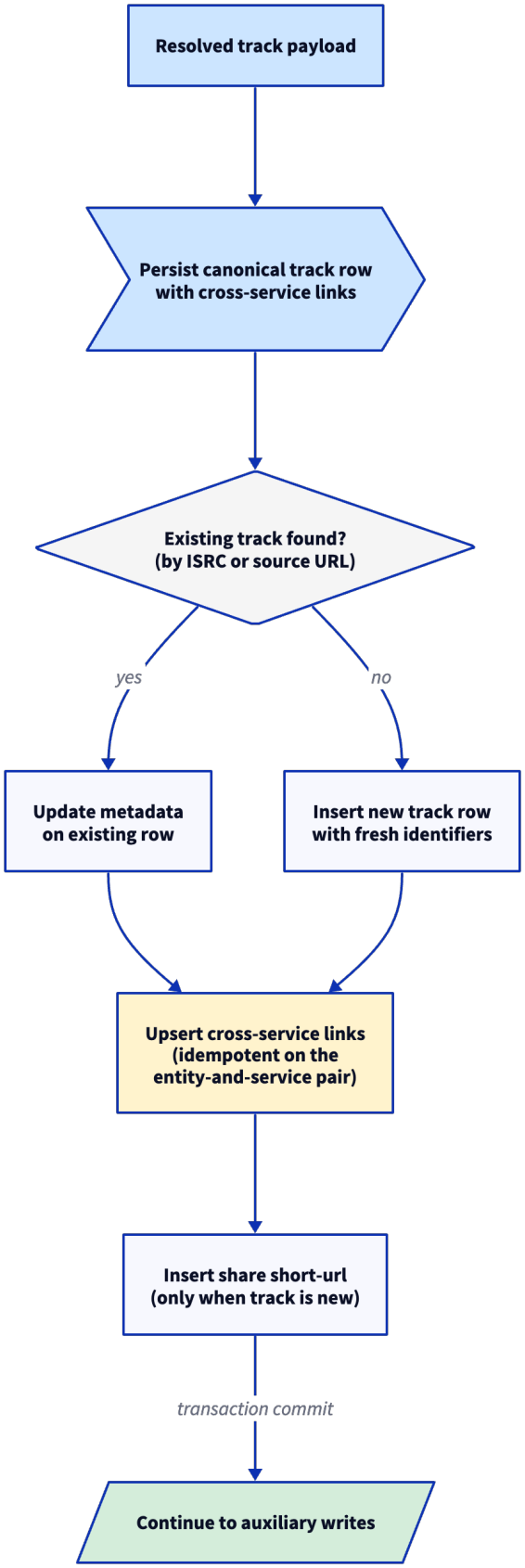

Figure 9: *Persistence: canonical write.*

9.2 Auxiliary writes and preview refresh

Every observed external identifier (ISRC, MBID[3], ISWC[4], and UPC) is stored in a dedicated aggregation table, inserted idempotently on the (entry, identifier type, value, source service) tuple. Failures at this step are not critical and are logged at debug level only; a stumbled aggregation write must not demote an otherwise successful resolve to a failure.

Each cross-service link that returned a preview URL is upserted with its respective expiry. The preview URL of the source service is stored the same way.

If the original input was a short link that had to be expanded, the resolver stores the original as an alias. Subsequent requests for the same short link hit the cache directly. A failure to write the alias has no consequences for the response; the alias is a pure optimisation.

If the preview URL is missing or expired and an ISRC is available, the resolver fetches a new URL from the most stable available source. As a result, the resolver still returns a playable preview after a cache hit, even when the previously stored one has expired in the meantime.

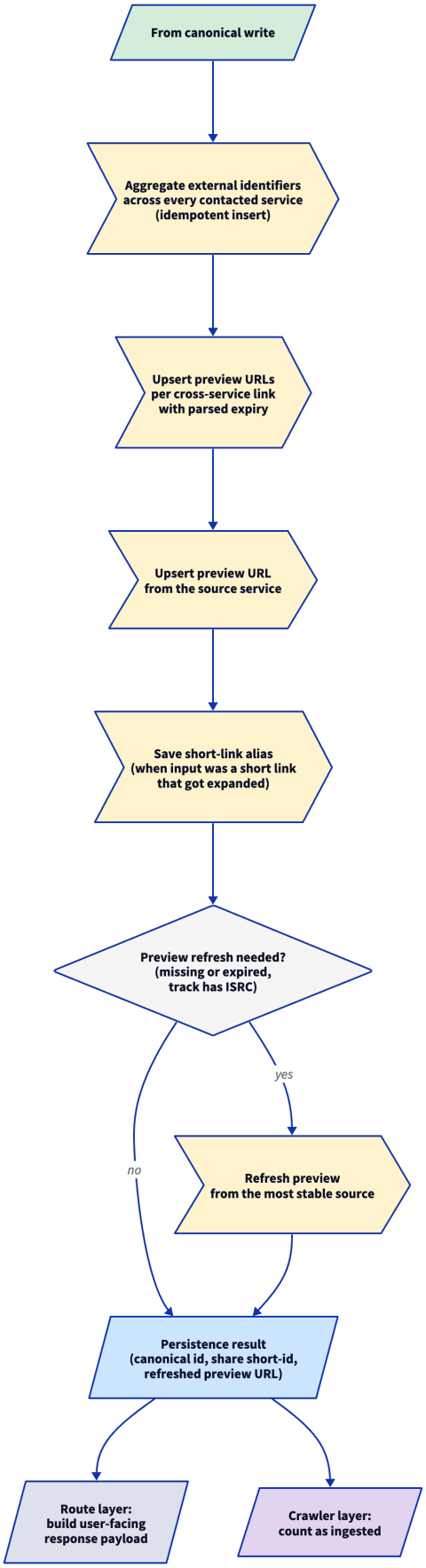

Figure 10: *Persistence: auxiliary writes and preview refresh.*

At the end the persistence layer returns three values to the caller: the canonical track identifier, the share short-link identifier, and the preview URL the caller passes on to the client. The route layer assembles the user-visible response from these; the crawler simply records the result as "ingested".

Album and artist persistence use their own helper functions in the route layer.

10 The cache strategy in one place

Every read-side cache access uses the canonical row that the persistence stage writes. Since the split into static and dynamic parts, canonical rows no longer expire. Once written, a track, album, or artist entry counts as current indefinitely. Only preview URLs carry an expiry timestamp and are refreshed lazily on read.

This split is what allows the resolver to add new services incrementally. When a new adapter is enabled today, the resolver fills in the missing link on the next request for an existing track; no cache reset is required. Freshness checks are scoped to the preview-URL table: they trigger a refresh only when no valid preview remains for the source in question. The same is true for the Deezer preview top-up, which runs only in this case. The fast cache-hit path therefore consists of a single indexed lookup.

11 Confidence thresholds

All thresholds live in a single central constants module and are re-exported from the resolver entry point as a convenience. The authoritative values reside in the module; the table below summarises what role each threshold plays.

Table 5: *Confidence thresholds and their roles.*

Constant	What it governs	Purpose
`MATCH_MIN_CONFIDENCE`	An adapter match must reach this score to count as a hit at all.	Filters obvious noise at the adapter boundary.
`LINK_QUALITY_THRESHOLD`	A cross-service link must reach this score to enter the response.	Keeps noisy fuzzy hits off the display.
`AUTO_SELECT_THRESHOLD`	A free-text search auto-resolves when the top candidate reaches this score.	Saves a follow-up request on clear cases.
`CANDIDATE_MIN_CONFIDENCE`	A candidate must reach this score to appear in the suggestion list.	Keeps hopeless options out of disambiguation.
`MAX_CANDIDATES`	Maximum suggestion-list length.	Caps response size.
`SEARCH_FALLBACK_CONFIDENCE`	Confidence assigned to synthetic search-fallback links.	Sorts such links to the end of the list.

The artist resolver uses its own confidence formula because it matches on name similarity rather than, like the track resolver, on identity of ISRC or text. The album resolver adopts the adapter-reported confidence when the strategy is text search; for matches by UPC or by ISRC inference, the values are fixed.

12 Design decisions at a glance

Several design decisions shape the behaviour of this layer in material ways:

Adapter timeout. The cross-service expansion runs each adapter call against an individual timeout. Without it, a single blocking adapter would hold up the whole response until the general HTTP timeout took effect, which lasts considerably longer.

Artwork fallback chain. Some services do not return artwork in their track API. When the source lacks artwork but an ISRC is known, the resolver fetches artwork from another service's match. Spotify and Apple Music are consistently high-quality in this regard and therefore sit at the front of the chain.

User-Agent for Open Graph extraction. Single-page applications return different content depending on User-Agent and expose Open Graph meta tags only to known crawlers. The Open Graph path therefore identifies itself as such a crawler in order to obtain the server-rendered variant.

Adapter order in free-text search. Adapters are walked in a fixed order. ISRC-rich sources with broad catalogue coverage (Deezer, Apple Music, Tidal) lead. Spotify drops to the end of the main chain and serves only as a fallback after ISRC misses. All further adapters form the safety net.

Static vs. dynamic split. Canonical rows have been permanently valid since the cache migration. Anything that can go stale lives in dedicated auxiliary tables with their own freshness rules. This split keeps the fast cache-hit path straightforward.

Thresholds in one place. All confidence values reside in a single module. The confidence behaviour is therefore traceable in one place, rather than scattered across the source.

Glossary

[1]ISRC, the International Standard Recording Code, is a twelve-character alphanumeric identifier assigned to a single recording. Because every streaming service uses the same ISRC for the same recording, it is the most reliable cross-service track identifier the resolver has at its disposal.

[2]UPC, the Universal Product Code, is a twelve-digit identifier in barcode form that names an album release at the industry level. Within the resolver, UPC plays the same role for albums as ISRC does for individual recordings.

[3]MBID, the MusicBrainz Identifier, is a UUID assigned by the MusicBrainz database. It identifies a recording, work, artist, or release at a finer granularity than ISRC and is unique per database entity.

[4]ISWC, the International Standard Musical Work Code, is an eleven-character identifier that names a musical work, not a particular recording of it. Multiple recordings of the same work share an ISWC even when they have different ISRCs.